GeLB Developer Manual

Release 1.0

Dragos-Bogdan Chirila

Nov 01, 2026

Contents

1 Repository map **2**
- 1.1 Top-level directories . . . 2
- 1.2 Top-level files . . . 3
- 1.3 The three commands . . . 3

2 The compilation pipeline **4**
- 2.1 Frontend . . . 4
- 2.2 Midware . . . 5
- 2.3 How a diagnostic is produced . . . 6

3 The AST and the visitors **7**
- 3.1 Nodes . . . 7
- 3.2 From the parse tree to the AST . . . 7
- 3.3 Visitors . . . 8
- 3.4 The pattern, in one existing visitor . . . 8

4 Backends **9**
- 4.1 The three generators . . . 9
- 4.2 Runtime headers . . . 10
- 4.3 How `gelb-run` compiles the generated code . . . 10
- 4.4 What the backends must agree on . . . 10

5 Extending GeLB **12**
- 5.1 Adding an intrinsic . . . 12
- 5.2 Adding a validation pass . . . 13
- 5.3 Adding a backend . . . 13

6 Testing **15**
- 6.1 The test tree . . . 15
- 6.2 Running the suite . . . 15
- 6.3 Markers and options . . . 15
- 6.4 The emission snapshot . . . 16
- 6.5 The numeric suite . . . 16
- 6.6 What CI runs . . . 16

7 Design decisions **17**

This manual describes the compiler for contributors: where the code is, how a program travels through it, and what to touch when adding to it. It describes GeLB as it is in the tree; every statement names the file, class, function or command that makes it true.

CHAPTER 1

Repository map

Every path is relative to the repository root; `git ls-files` lists the tracked files.

1.1 Top-level directories

`gelb/`
: The compiler package. gelb/gelbc.py is the `gelbc` driver, gelb/compiler_pipeline.py orders the passes (*The compilation pipeline*) and gelb/run_cli.py is the `gelb-run` entry point, which hands over to the runner gelb/scripts/gelb-run. The sub-packages follow the three stages: gelb/frontend/ (parse tree to AST, symbol tables, types), gelb/midware/ (constant folding, stencils, geometry, the rewrites) and gelb/backends/ (the three generators and their runtime headers, *Backends*). gelb/gelb_ast/ holds the node classes, gelb/antlr_part/ the grammar and the generated parser, gelb/utils/ diagnostics and configuration, gelb/visualization/ the `gelb-view` tool, gelb/debugging/ the AST drawing behind `gelbc --visualize-ast`, and gelb/tests/ the suite (*Testing*). In pyproject.toml, `[tool.setuptools.packages.find]` keeps `gelb.tests*` out of the wheel and `package-data` ships the `*.hpp` and `*.cuh` runtime headers and the `gelb-run` runner gelb/scripts/gelb-run with it.

`examples/`
: The `.gelb` programs, in `basic/`, `fd/`, `lbm/` (`1d`, `2d`, `3d`), `lgca/`, `sukop/` and `softwarex-2026-paper/`. The worked chapters of the user tutorial are each anchored by one of them, the emission snapshot compiles all of them (*Testing*), and examples/VALIDATION.md describes the analytical validation that scripts/validate_analytical.py runs.

`docs/`
: The Sphinx project: docs/conf.py, docs/index.rst, the user tutorial under docs/user/ rooted at docs/usrman.rst, this manual under `docs/dev/` rooted at docs/devman.rst, the Pygments lexer for `.. code-block:: gelb` in docs/_ext/gelb_lexer.py, and docs/figs/. docs/LANGUAGE_DESIGN.md records language design decisions and is excluded from the build by `exclude_patterns` in docs/conf.py.

`scripts/`
: Developer tools. This manual relies on scripts/gelb-run (a shim for the `gelb-run` runner gelb/scripts/gelb-run, which compiles and runs a program, *Backends*), scripts/emission_snapshot.py (byte-compare the generated code of every example), scripts/regenerate_antlr.sh (rebuild the parser), scripts/gelb_bootstrap.sh (create the `uv` environment and, through scripts/gelb-shell-setup, put its `.venv/bin` on `PATH`),

scripts/build_manual_pdfs.py with scripts/build_usrman_pdf.py and scripts/build_devman_pdf.py (the release PDFs), scripts/clean_artifacts.sh (remove build output) and scripts/install-vscode-ext.sh. The `bench*`, `compare_backends.py` and `validate_*.py` scripts measure and validate simulations.

`vscode-gelb/`
: The VS Code extension: the TextMate grammar vscode-gelb/syntaxes/gelb.tmLanguage.json, the snippets vscode-gelb/snippets/gelb.json, the TypeScript sources under vscode-gelb/src/ and the tests under vscode-gelb/test/: the Node tests run with `npm test`, the VS Code-hosted ones in `test/suite/` with `npm run test:vscode`. `extension.ts` calls `gelbc --diagnostics-json` to fill the Problems panel.

`third_party/`
: `highway/` is the Google Highway SIMD library, a git submodule (.gitmodules) that `gelb-run` puts on the include path when it compiles for the `cpu_highway` backend.

1.2 Top-level files

pyproject.toml defines the package, its dependencies and the extras `gui` (PySide6), `docs` (Sphinx, furo, copybutton) and `dev` (pytest and friends); `uv sync --all-extras` installs them all, as CONTRIBUTING.md says. .readthedocs.yaml points ReadTheDocs at docs/conf.py; .github/workflows/release-health.yml is the only CI workflow (*Testing*).

1.3 The three commands

`[project.scripts]` in pyproject.toml defines three console entry points: `gelbc = "gelb.gelbc:cli"`, `gelb-run = "gelb.run_cli:main"` and `gelb-view = "gelb.visualization.gelb_view:cli"`. `gelbc` and `gelb-view` are `click` commands; `gelb-view` is a group whose subcommands are `data` (a NetCDF output file), `algo` (the stencils of a program), `geo` (its node-class map) and `source`; a first argument that is not a subcommand name is routed to `source` when it ends in `.gelb` and to `data` otherwise, and with no arguments the group opens the GUI. `gelb-run` is a bash script, the runner gelb/scripts/gelb-run, shipped as package data: gelb/run_cli.py replaces its own process with that script (`os.execv`), passing the arguments and the environment through unchanged, so that the command is the same plain word in a checkout and in a packaged installation. scripts/gelb-run is a ten-line shim that `exec`s the runner of the tree it sits in, `<checkout>/gelb/scripts/gelb-run`; the macOS bundle's `gelb-run` wrapper and the test suite call it by that path. The runner starts `gelbc` through the interpreter named by `GELB_PYTHON` (`python -m gelb.gelbc`), otherwise through `uv run --project <checkout>` and, when `uv` is not installed either, through the checkout's `.venv/bin/python`; it compiles the result with the native compiler of the chosen backend, runs the executable and, with `--view`, hands the output to `gelb-view` (*Backends*). `uv sync` installs the three commands in `.venv/bin`; scripts/gelb_bootstrap.sh ends by calling scripts/gelb-shell-setup with that directory, which appends a marker comment and one `PATH` line to the startup file of each of the user's shells (for bash on Linux, to `~/.bashrc` and also to the login file when that does not already source `~/.bashrc`), and never a second time.

CHAPTER 2

The compilation pipeline

`compile_source` in gelb/compiler_pipeline.py is the one place that orders the passes. `gelbc` calls it (through `compile_file`) with the `CODEGEN` profile, `gelbc --diagnostics-json` with `DIAGNOSTICS`, and the `gelb-view` analysis modes (through gelb/midware/program_analysis.py) with `PROGRAM_ANALYSIS`; the three are the `PipelineProfile` enum. Every pass is followed by `record(name)`, so `CompilationResult.pass_names` is the executed sequence, and gelb/tests/unit/test_compiler_pipeline.py asserts it per profile. The frontend passes are wrapped by the functions of gelb/frontend/stage1_frontend.py and the midware checks by those of gelb/midware/stage2_midware.py; `apply_constant_overrides`, `sequence_draws`, `inline_functions`, `hoist_intrinsic_args`, the three rewrites and `resolve_geometry` are imported straight from their own modules. The module behind each pass is named below, under the names `record` uses. Two sanity checks: `validate_ast_root` confirms that the root is a `Program_N`, and `validate_midware_input` confirms that again and that the root carries the seven block attributes (`global_constants` through `geometry`).

Not every pass runs in every profile: `detect_function_recursion` and the midware passes from `validate_streaming_intrinsic_args` to `slice_df_expansion` run under `CODEGEN` and `DIAGNOSTICS` only; `extract_stencils` and the checks after it run only with `PipelineOptions.require_stencils`, the map-based checks only with `require_geometry`, and `geometry_lint` not under `--no-geometry-lint`; `PROGRAM_ANALYSIS` runs `resolve_geometry` (with `require_geometry`) and then `extract_stencils` (with `require_stencils`) and none of the coverage, wall, write-destination or lint checks (gelb/tests/unit/test_compiler_pipeline.py).

2.1 Frontend

`parse`
: `get_ast`: the lexer and parser from gelb/antlr_part/generated_code/ build the parse tree, gelb/frontend/ParsingError.py turns a syntax error into `GeLBExSyntaxError`, and gelb/frontend/ASTBuilderVisitor.py builds the AST.

`validate_lattice_access_context`
: gelb/frontend/LatticeAccessContextValidator.py: `f.out` only in initializers and dynamics, `f.in` only in dynamics and gauges, neither in `global_constants`.

`validate_block_signatures`
: gelb/frontend/GaugeValidationVisitor.py: functions and gauges declare named results, and a gauge assigns all of

its outputs.

`prevent_symbol_ref_implicit_clashes`
: gelb/frontend/ImplicitsRefsClashPreventerVisitor.py: the implicit symbols (`x`, `y`, `z`, `t`, `Nx`, `Ny`, `Nz`) are not referenced in blocks where they do not apply, and the coordinates are not assigned to.

`build_symbols_tables`, `prevent_symbol_def_implicit_clashes`
: gelb/frontend/SymbolsTablesBuilderVisitor.py attaches a `SymbolsTable` (gelb/frontend/SymbolsTable.py) to the program and to each block, after the wrapper has checked every lattice's grid-size rank against its `d`; then gelb/frontend/ImplicitsDefsClashPreventerVisitor.py: no user symbol is named like an implicit one that applies in its block (`reserved_definition_names_for_block` in gelb/frontend/implicit_symbols.py; a function may name a local `x` or `t`, but not `Nx`).

`detect_function_recursion`
: in `stage1_frontend.py` itself: a depth-first search over the user-function call graph.

`add_implicit_symbols`
: gelb/frontend/ImplicitSymbolsAdderVisitor.py adds the dimension constants (`Nx`, and `Ny`, `Nz` as the dimensionality allows) to the program's table and the coordinates and `t` to each initializer, dynamic and gauge; the geometry block gets the coordinates but not `t`.

`validate_fixed_geometry`, `apply_constant_overrides`
: gelb/frontend/GeometryDependencyValidator.py: paints depend only on coordinates and constants; then gelb/frontend/constant_overrides.py, only with `-D NAME=VALUE`.

`validate_global_constant_initializers`
: gelb/frontend/GlobalConstantsInitializerValidator.py: no `random()` or user function in a global constant's initializer.

`resolve_all_constants`, `resolve_ranges`
: gelb/frontend/ConstantsResolverVisitor.py substitutes explicit and implicit constants into their uses; gelb/frontend/RangeResolverVisitor.py fills omitted range bounds with `0` and `Nx-1` (`Ny-1`, `Nz-1`).

`infer_types`, `promote_types`, `validate_types`
: gelb/frontend/TypeInferenceVisitor.py sets `inferred_type` on every expression, gelb/frontend/TypePromotionVisitor.py inserts `TypePromotionNode_N` where a conversion is implicit, gelb/frontend/TypeValidationVisitor.py rejects what is left; the first two ask gelb/frontend/type_rules.py.

`validate_constant_assignments`, `validate_lattice_consistency`
: gelb/frontend/ConstantAssignmentValidationVisitor.py: no assignment to a constant; gelb/frontend/LatticeConsistencyValidator.py: all lattices share one grid size and periodicity.

`validate_node_classes`, `initial_validation_geometry`
: gelb/frontend/NodeClassesValidationVisitor.py and gelb/frontend/GeometryInitialValidationVisitor.py: the blocks a node class names, and the node classes a paint names, exist.

2.2 Midware

`fold_constants`, `validate_division_by_zero`, `lower_local_constants`
: gelb/midware/ConstantFoldingVisitor.py folds, evaluating with gelb/midware/ast_eval.py; gelb/midware/DivisionByZeroValidator.py runs after it so that a folded zero is caught; gelb/midware/LocalConstantLowering.py rewrites a block-local `const` as a typed declaration.

`validate_streaming_intrinsic_args`, `sequence_draws`
: gelb/midware/StencilArgValidator.py checks the velocity sets passed to `upstream_dfs` and `bounce_back_dfs`; gelb/midware/DrawSequencing.py moves each `random()` call into its own temporary, in source order.

inline_functions, hoist_intrinsic_args
gelb/midware/inline_functions.py replaces every user function call by the function body; gelb/midware/IntrinsicArgHoisting.py gives `dot_prod` bare identifiers as arguments.

collinearity_rewrite, scalar_block_cse, slice_df_expansion
gelb/midware/CollinearityRewriteVisitor.py (off unless `PipelineOptions.enable_collinearity_rewrite` is set, which `gelbc` never does), gelb/midware/ScalarBlockCSEVisitor.py (`--no-scalar-cse` turns it off) and gelb/midware/SliceDFExpansionVisitor.py (`--no-slice-simplify` turns it off), the last two simplifying through gelb/midware/ast_simplify.py (*Design decisions*).

extract_stencils, validate_initializer_coverage, warn_writeonly_dynamics, warn_unused_lattices
gelb/midware/StencilExtractionVisitor.py collects the lattice reads and writes of every block into a `ProgramStencils` (gelb/midware/stencil.py); `stage2_midware.py` then checks that an initializer that writes a lattice writes all of its DFs, and emits two warnings.

resolve_geometry, validate_boundary_access, validate_write_destinations, validate_df_coverage, geometry_lint
gelb/midware/geometry.py resolves the paints into the per-cell node-class map; on that map `stage2_midware.py` checks reads and writes off a wall, the destinations of writes at an offset, and that every DF a block reads is written in time; then gelb/midware/geometry_lint.py warns. `gelbc` validates `--output-mode` first and gets the same checks from `resolve_geometry_for_compile`.

2.3 How a diagnostic is produced

Every error is a subclass of `GeLBException` (gelb/utils/exceptions.py; one more subclass sits in gelb/frontend/LatticeConsistencyValidator.py), constructed with an `error` and a `hint`; twelve of the twenty-five subclasses in `exceptions.py` also carry a source location (the offending node or token, or a source line) that `_source_location` returns as a line and column. The constructor prints the exception at once: `__str__` calls `show_error_in_src` from gelb/utils/error_utils.py, which prints the source lines around the location (`GELB_ERROR_CONTEXT = 2` in gelb/utils/config.py on either side), a caret under the column, `[Problem] <error>` in red and `[Hint] <hint>` in green, and then `os._exit(1)`; without a location it prints the two banners alone (`error_with_hint_to_term` in gelb/utils/output_to_terminal.py). `gelbc` sets `sys.tracebacklimit = 0` unless `-t` is given, so no traceback follows. Warnings go to stderr with a `[Warning]` label.

Under pytest and in diagnostics mode, `_suppress_print_and_exit` in `exceptions.py` suppresses the printing and the exit. `_run_diagnostics` in gelb/gelbc.py sets `config.DIAGNOSTICS_MODE`, runs the `DIAGNOSTICS` profile with stencils and geometry required, locates the one `GeLBException` it catches with `resolve_error_location` (`error_utils.py`, 1-based lines) and turns that into a 0-based range with `_diag_range` (any other exception becomes a file-level `(internal)` error), appends the warnings that `warning_with_hint_to_term` collected in `config.DIAGNOSTICS_WARNINGS`, writes one JSON document `{"diagnostics": [...]}` to stdout and exits 1 when an error is among them.

CHAPTER 3

The AST and the visitors

3.1 Nodes

Every AST node class lives in gelb/gelb_ast/ASTNodes.py and derives from `Node_N`, an abstract class whose constructor takes a `code_range` (a `CodeRange` from gelb/utils/CodeRange.py: 1-based `start_line` and `stop_line`, 0-based `start_col` and `stop_col`). `Node_N` has no field list: `get_children_names` discovers a node's children with `dir()`, skipping private names, `code_range`, `symbol_defs` and the metadata attributes that later passes attach (`_AST_METADATA_FIELD_NAMES`: `inferred_type`, `symbol_tag` and the others) and the names of `Node_N`'s own methods, so a new public attribute, and a new public method, on a node is a child unless it is listed there; give helper methods a leading underscore. On top of that come `get_children`, `get_referenced_symbols` (the `SymbolReference` objects of gelb/gelb_ast/SymbolReference.py reached from a subtree), `replace_symbol_references` and `is_terminal_node`. `List_N` wraps a Python list of nodes. `Program_N` is the root, with one attribute per top-level construct (`simulation_description`, `global_constants`, `lattices`, `initializers`, `dynamics`, `gauges`, `functions`, `node_classes`, `geometry`); the root and the blocks carry a `symbol_defs` table once `build_symbols_tables` has run (`SymbolsTable` in gelb/frontend/SymbolsTable.py, a list of `SymbolDefinition` from gelb/gelb_ast/SymbolDefinition.py with a parent table). `Stmt_N`, `Expr_N` and `Block_N` are the intermediate bases; `Div_N` has `left` and `right`, `IntrinsicFunctionCall_N` has `function_name` (a string) and `arguments` (a `List_N`).

3.2 From the parse tree to the AST

gelb/antlr_part/grammars/GeLB.g4 (with the tokens of gelb/antlr_part/grammars/CommonLexerRules.g4) is the grammar; `GeLBLexer.py`, `GeLBParser.py` and `GeLBVisitor.py` in gelb/antlr_part/generated_code/ are generated from it by `invoke_antlr4_tool` in gelb/utils/misc.py, which scripts/regenerate_antlr.sh runs. `get_ast` in gelb/frontend/stage1_frontend.py parses the source with the start rule `gelb_program`, then hands the parse tree to `ASTBuilderVisitor` (gelb/frontend/ASTBuilderVisitor.py), a subclass of the generated `GeLBVisitor`. Its `visitGelb_program` creates the `Program_N` and sorts the children returned by `get_list_of_children_sub_asts` into the root's attributes; that helper visits every child of a rule context and keeps the results that are truthy (`None` is dropped, and so is an empty `List_N`, because `List_N` defines `__len__`), so a rule with no `visit...` override contributes nothing to the AST. The builder counts the blocks it meets and raises `GeLBExInvalidNoOfOccurrences` against the multiplicities written as comments on the `gelb_program`

rule in `GeLB.g4`. Every node the builder creates gets its `code_range` from `code_range_from_ctx` in `CodeRange.py`, except the four `List_N` containers on the root for the repeatable blocks (`initializers`, `dynamics`, `gauges`, `functions`), whose `code_range` is `None` because their members need not be contiguous in the source.

3.3 Visitors

`ASTBaseVisitor` in gelb/frontend/ASTBaseVisitor.py has one method, `visit(node, ast_root_node)`: it looks up `visit<ClassName>` on itself, calls it with the node and the root, and raises `GeLBExMissingASTVisitationMethod` if the method is missing; the root is passed everywhere so that no visitor needs a global. Most frontend passes and the rewriting midware passes of *The compilation pipeline* are such visitors (the classes in gelb/frontend/ and gelb/midware/ that derive from `ASTBaseVisitor`); the remaining passes, among them `detect_function_recursion`, `lower_local_constants`, `sequence_draws`, `inline_functions`, `hoist_intrinsic_args`, `resolve_geometry` and the stencil checks after `extract_stencils`, are plain functions, or walkers of their own (`StencilExtractionVisitor`, `LatticeAccessContextValidator`), over the AST or over `ProgramStencils`. gelb/frontend/visitor_mixins.py supplies the shared parts: `BinaryOperationMixin` (one handler for all binary operators), `ArrayLiteralProcessingMixin`, `ProgramTraversalMixin` (`traverse_program_blocks` visits the blocks in a fixed order and calls the `_before_block_visit` and `_after_block_visit` hooks; `_visit_block_stmts` visits a block's `stmts`) and `SymbolTableUtilityMixin`.

3.4 The pattern, in one existing visitor

gelb/midware/DivisionByZeroValidator.py is a complete small pass. `class DivisionByZeroValidator(ASTBaseVisitor, ProgramTraversalMixin)`; `visitProgram_N` calls `traverse_program_blocks`; the block methods call `_visit_block_stmts`; `visitDiv_N` visits `left` and `right` and raises `GeLBExTypeError(node.right, error, hint)` when the divisor is a literal zero (`0`, `0.0` or `false`), possibly under a `TypePromotionNode_N`; `visitIntrinsicFunctionCall_N` does the same for `mod` with a zero second argument and `visitPow_N` for an integer `0` raised to a negative integer literal; every other expression method visits its children, and the terminal methods (`visitIntLiteral_N` and the like) return the node. The wrapper `validate_division_by_zero` in gelb/midware/stage2_midware.py instantiates it and calls `visit(ast, ast)`; `compile_source` calls the wrapper and `record("validate_division_by_zero")`; gelb/tests/unit/midware/test_division_by_zero_validation.py tests it and gelb/tests/unit/test_compiler_pipeline.py pins its place in the sequence (*Extending GeLB*).

CHAPTER 4

Backends

`gelbc --backend` accepts `cpu_fortran_serial`, `cpu_highway` (the default) and `gpu_cuda_cpp` (the `click.Choice` in gelb/gelbc.py). Each generator takes the AST, the run parameters and the `BackendContext` from `build_backend_context` in gelb/compiler_pipeline.py, and returns the target program as one string, which `gelbc` writes to `-o` or to the input name with the backend's extension.

4.1 The three generators

`cpu_fortran_serial`
: `generate_fortran` and `FortranCodeGenerator` in gelb/backends/stage3_backend_cpu_fortran_serial.py emit a single-threaded Fortran 90 program (`.f90`). gelb/backends/fortran/common.py holds the timing and throughput report lines and gelb/backends/fortran/line_formatter.py splits lines longer than 132 characters at token boundaries. This is the only backend that supports `--output-mode stdout` (checked in `gelbc.py` and in `gelb/scripts/gelb-run`).

`cpu_highway`
: `generate_cpu_highway` and `HighwayCodeGenerator` in gelb/backends/stage3_backend_cpu_highway.py emit a C++17 program (`.cpp`) with OpenMP loops and Google Highway SIMD. The class is assembled from the mixins of gelb/backends/highway/: `expr.py` (expressions and intrinsic calls), `statements.py` (statements, lattice reads and writes, the streaming intrinsics), `simd.py` (vector bodies, peeled edges and scalar tails), `layout.py` (storage layout, allocation, halo updates), `output.py` (NetCDF and gauges) and `common.py` (type maps, identifier encoding, host probes).

`gpu_cuda_cpp`
: `generate_cuda_cpp` and `CudaCppCodeGenerator` in gelb/backends/stage3_backend_gpu_cuda_cpp.py emit CUDA C++ (`.cu`). The class subclasses `HighwayCodeGenerator` with `use_openmp=False` and `storage_layout='soa'` and adds the mixins of gelb/backends/cuda/: `device.py` (device storage and cuRAND state), `kernels.py` (the `__global__` kernels), `launch.py` (launch configuration, the time loop, CUDA graphs) and `tiling.py` (`--cuda-tiled`, `--cuda-ldg`, `--cuda-stwt`). Expression and statement emission are inherited, so a change in `highway/expr.py` reaches the CUDA output.

Shared by more than one generator: gelb/backends/gauge_outputs.py (the gauge fields of the NetCDF file) and gelb/backends/string_literals.py (quoting) by all, and gelb/backends/real_literals.py by the C++ ones.

4.2 Runtime headers

The C++ programs include a header (`runtime_header` on each generator class, emitted by `_emit_includes`) rather than carrying their helpers inline. gelb/backends/cpu_highway_runtime.hpp and gelb/backends/gpu_cuda_cpp_runtime.cuh both include gelb/backends/cpp_runtime_common.hpp, which holds the NetCDF wrappers and the scalar reduction and `min`/`max` helpers; the Highway header adds the vector overloads, the CUDA header the `CUDA_CHECK` macro and the event timer. The Fortran program is self-contained. With `--strict-fp`, `_emit_includes` writes `#define GELB_STRICT_FP 1` before the include, and `cpu_highway_runtime.hpp` then defines `gelb_mul_add` as a separate multiply and add instead of `hn::MulAdd`.

4.3 How `gelb-run` compiles the generated code

gelb/scripts/gelb-run translates with `gelbc` (`env VIRTUAL_ENV= uv run --project <checkout> gelbc`, or `$GELB_PYTHON -m gelb.gelbc` when `GELB_PYTHON` is set, which the script does itself from the checkout's `.venv/bin/python` when `uv` is not found), compiles with the `compile_src` of the chosen backend's `case` branch, runs the executable (behind gelb/visualization/progress_monitor.py when a NetCDF run has more than one time step) and removes the intermediates unless `--keep` is given. The native compilers and their defaults:

- `cpu_fortran_serial`: `GELB_FC` (default `gfortran`) with `GELB_FFLAGS` (default `-O3 -march=native -ffp-contract=off -fno-range-check -ffree-line-length-none`) and the NetCDF-Fortran flags `GELB_NETCDF_INC` and `GELB_NETCDF_LIB` (detected with `nf-config` and `nc-config` when unset).
- `cpu_highway`: `GELB_CXX` (default `g++` on Linux; on macOS the first of `g++-16` to `g++-13`, else `clang++` with the OpenMP runtime from `GELB_OMP_ROOT` or Homebrew's `libomp`) with `GELB_CXXFLAGS` (default `-std=c++17 -O3 -march=native -ffp-contract=off -fno-trapping-math -fno-unroll-loops -DNDEBUG`, with `-mcpu=native` in place of `-march=native` for Apple clang on arm64; OpenMP flags appended), `-I` for `third_party/highway` and `gelb/backends`, four Highway support `.cc` files compiled in, and NetCDF-C from `GELB_NETCDF_C_ROOT` or `GELB_NETCDF_C_INC` and `GELB_NETCDF_C_LIB` (`nc-config` when unset). At run time it fills in `OMP_NUM_THREADS` (physical cores) and, on Linux, `OMP_PROC_BIND` and `OMP_PLACES`, each only when unset.
- `gpu_cuda_cpp`: `GELB_NVCC` (default `nvcc`) with `GELB_NVCC_ARCH` (default `-arch=sm_89`) and `GELB_NVCC_FLAGS` (default `-O3 $NVCC_ARCH -std=c++17 --expt-relaxed-constexpr -DNDEBUG -diag-suppress=177`), plus the same NetCDF-C variables.

`-F` appends the compiler's fast-math flag; `--strict-fp` passes `--strict-fp` to `gelbc`, appends `-ffp-contract=off` (CPU) or `-fmad=false` (nvcc), and rejects `-F` and any override carrying a value-changing flag (`reject_value_changing_flags`). `GELBC_EXTRA` forwards extra `gelbc` options. `gelbc` itself reads `GELB_CXX`, `GELB_CXXFLAGS` and `GELB_HAS_AVX512` to detect AVX-512, which decides the `cpu_highway` storage layout (*Design decisions*).

4.4 What the backends must agree on

The user tutorial's chapter *Numerical Semantics* (docs/user/19_numerical_semantics.rst) is the authoritative statement; its table *What must agree* says which results are bitwise identical between a default and a plain build, and across backends in strict and in default builds: the IEEE basic operations (in strict builds), folded constants, `min`, `max`, `mod` and integer powers agree across backends, while real powers, reductions and library functions may differ. gelb/tests/numeric/ enforces it against an independent C++ oracle: `cases.py` generates the corpus, gelb/tests/numeric/conftest.py compiles the oracle with `-O0 -fno-fast-math -ffp-contract=off`, `test_generated_constants.py` checks the folded constants, and `test_generated_runtime.py` builds every runtime case (those not marked `runtime=False` in `cases.py`) on all three backends through `gelb-run` in an `optimized` and a

`plain` mode (`MODES`) and compares bit for bit (`checks.py`). gelb/tests/integration/test_real_width_contract.py, gelb/tests/integration/test_reproducibility.py and gelb/tests/integration/test_math_library_accuracy.py cover the build modes, reproducibility across execution configurations and library accuracy, as the chapter's notes name them.

CHAPTER 5

Extending GeLB

Each checklist names the files one existing feature touches. Run the checks of *Testing* afterwards; a change not meant to alter the generated code must leave the emission snapshot identical.

5.1 Adding an intrinsic

`bounce_back_dfs` is the model. It passes through:

1. **Grammar.** The token `BOUNCE_BACK_DFS : 'bounce_back_dfs' ;` in gelb/antlr_part/grammars/CommonLexerRules.g4 and the alternative in the rule `intrinsic_function_name` of gelb/antlr_part/grammars/GeLB.g4. Run scripts/regenerate_antlr.sh and commit the regenerated files of gelb/antlr_part/generated_code/ in the same commit (CONTRIBUTING.md).
2. **AST.** `visitIntrinsicFunctionCallExpr` in gelb/frontend/ASTBuilderVisitor.py maps the token to the string `"bounce_back_dfs"` and builds an `IntrinsicFunctionCall_N`; no new node class is needed.
3. **Type rule.** The entry in `INTRINSIC_FUNCTIONS` of gelb/frontend/intrinsic_functions.py names the argument checker (`_check_streaming_dfs_arguments`) and the return-type computer (`_compute_streaming_dfs_return_type`); `TypeInferenceVisitor.visitIntrinsicFunctionCall_N` looks the entry up (a streaming intrinsic also goes into the `_STREAMING_INTRINSICS` set of that method, which skips the lattice-name argument and fixes the `array[real, q]` result) and `TypePromotionVisitor.visitIntrinsicFunctionCall_N` converts the arguments. A scalar intrinsic also goes into `SCALAR_INTRINSICS`, `intrinsic_argument_types` (the conversions `TypePromotionVisitor` applies) and `intrinsic_result_type` of gelb/frontend/type_rules.py.
4. **Constant folding.** `_try_evaluate_intrinsic` in gelb/midware/ConstantFoldingVisitor.py dispatches to `intrinsic` and `_LIBRARY` in gelb/midware/ast_eval.py for scalar intrinsics and returns `None` for the streaming ones, which are never folded.
5. **Midware.** `_STENCIL_INTRINSICS` in gelb/midware/StencilArgValidator.py (argument validation), the `function_name` test in gelb/midware/StencilExtractionVisitor.py (the reads it implies), `_KNOWN_ARRAY_INTRINSICS` in gelb/midware/SliceDFExpansionVisitor.py, and `visitIntrinsicFunctionCall_N` in gelb/midware/DivisionByZeroValidator.py for an intrinsic with a divisor.

6. **Backends.** Fortran: `_emit_intrinsic_call` and `_emit_streaming_dfs` in gelb/backends/stage3_backend_cpu_fortran_serial.py. Highway: `_emit_intrinsic_call` and `_emit_intrinsic_bounce_back_dfs` in gelb/backends/highway/expr.py, `_collect_bounce_back_dfs_keys` and `_emit_bounce_back_dfs_tables` in gelb/backends/highway/statements.py, the table cache, the call of the tables emitter and the eligibility walk in gelb/backends/stage3_backend_cpu_highway.py, and the SIMD case in gelb/backends/highway/simd.py. CUDA inherits these (nothing under gelb/backends/cuda/ names the intrinsic).
7. **Tests.** gelb/tests/unit/parsing/test_parse_intrinsic_functions.py, gelb/tests/unit/type_inference/test_intrinsic_function_type_inferer gelb/tests/unit/midware/test_intrinsic_folding.py, gelb/tests/unit/midware/test_stencil_arg_validation.py, gelb/tests/unit/midware/test_streaming_intrinsic_stencils.py, gelb/tests/unit/midware/test_slice_df_expansion.py, gelb/tests/unit/midware/test_bounce_back_dfs.py and gelb/tests/integration/test_bounce_back_dfs.py (every backend, through `gelb-run`).
8. **Documentation.** The section *Built-in intrinsics* of docs/user/03_gelb_in_10_minutes.rst, the matching section of docs/LANGUAGE_DESIGN.md (*Lattice operations* for the streaming ones), the `INTRINSIC_FUNCTIONS` tuple of docs/_ext/gelb_lexer.py (syntax highlighting in the manual) and, if the numerical semantics change, docs/user/19_numerical_semantics.rst.
9. **VS Code.** The `intrinsic-functions` pattern of vscode-gelb/syntaxes/gelb.tmLanguage.json and a snippet in vscode-gelb/snippets/gelb.json; `npm test` in `vscode-gelb/` checks the grammar.

5.2 Adding a validation pass

1. A module in gelb/frontend/ or gelb/midware/ with a visitor derived from `ASTBaseVisitor` (and `ProgramTraversalMixin`), on the pattern of gelb/midware/DivisionByZeroValidator.py (*The AST and the visitors*).
2. An error class in gelb/utils/exceptions.py that takes the offending node and returns its location from `_source_location`, or an existing one such as `GeLBExTypeError`. A warning instead returns `(message, hint, source_line)` tuples fed to `emit_warning` in `compile_source`, like `warn_unused_lattices`.
3. A wrapper in gelb/frontend/stage1_frontend.py or gelb/midware/stage2_midware.py, and the call plus `record(name)` at its place in `compile_source` of gelb/compiler_pipeline.py.
4. The expected sequences in gelb/tests/unit/test_compiler_pipeline.py, tests under gelb/tests/unit/validation/ or gelb/tests/unit/midware/ written with the helpers of gelb/tests/test_utils.py (`minimal_gelb_program_1d`, `get_typed_ast` and the `assert_*_error` functions), and, if the diagnostics of an example change, gelb/tests/integration/test_example_diagnostics.py.

5.3 Adding a backend

1. `gelb/backends/stage3_backend_<name>.py` with a `generate_...(ast, ..., backend_context=...)` function returning the program text (`generate_fortran`, `generate_cpu_highway` and `generate_cuda_cpp` are the existing ones; `gelbc` imports each by name); a runtime header next to the others is shipped by the `package-data` globs of pyproject.toml (`*.hpp` and `*.cuh` under `gelb.backends`), so a header with another extension needs its own glob there.
2. The `click.Choice` of `--backend` and the dispatch branch in gelb/gelbc.py.
3. A `case` branch in gelb/scripts/gelb-run (`EXT` and `compile_src`), its validation list and the text of `_die_unknown_backend`.
4. `BACKENDS` in scripts/emission_snapshot.py; a toolchain guard in gelb/tests/integration/_toolchain.py; the backend list of gelb/tests/integration/test_examples_smoke.py (its own names: `serial` stands for `cpu_fortran_serial`) and the `--smoke-backends` option of

gelb/tests/integration/conftest.py; the backend lists of gelb/tests/integration/test_tutorial_regressions.py and gelb/tests/numeric/test_generated_runtime.py, so that the numeric contract is enforced on it.

5. docs/user/16_performance_backends.rst, the *What must agree* table of docs/user/19_numerical_semantics.rst, the toolchain list in CONTRIBUTING.md and, if CI is to build it, the `apt` line of .github/workflows/release-health.yml.
6. **VS Code.** The `Backend` type and the `BACKENDS` picker list of vscode-gelb/src/runCommand.ts, and vscode-gelb/test/runCommand.test.js.

CHAPTER 6

Testing

6.1 The test tree

Everything is under gelb/tests/: `unit/` (grouped by stage: `lexing`, `parsing`, `ast_building`, `symbols`, `data_types`, `type_inference`, `type_promotion`, `type_validation`, `validation`, `middleware`, and an empty `debugging` package, plus top-level modules such as gelb/tests/unit/test_compiler_pipeline.py), `integration/` (end-to-end runs through `gelbc` and `gelb-run`), `numeric/` (the conformance battery, *Backends*), `fixtures/` (`.gelb` programs for the regression and hardening cases), `gelb_snippets/` and `data/`. gelb/tests/test_utils.py holds the shared helpers and the minimal-program generators. gelb/conftest.py regenerates the parser with `invoke_antlr4_tool()` in `pytest_sessionstart`, so a test run needs a Java runtime.

6.2 Running the suite

`uv run pytest` from the repository root collects all of `gelb/tests`; `uv run pytest -q gelb/tests/unit` is the fast subset. Tests that need a native toolchain skip themselves: `requires_compiler` (gfortran and NetCDF-Fortran), `requires_cpp_highway_compiler` (g++, NetCDF-C and the Highway submodule) and `requires_gpu_cuda_cpp_compiler` in gelb/tests/integration/_toolchain.py; the numeric runtime test and gelb/tests/integration/test_real_width_contract.py gate their Fortran cases on gfortran and NetCDF-C instead; and the `numeric_oracle` fixture in gelb/tests/numeric/conftest.py needs a C++17 compiler. `uv run pytest -rs` prints the reasons.

6.3 Markers and options

- `gpu` is registered in `[tool.pytest.ini_options]` of pyproject.toml and marks the CUDA cases of the numeric suite, gelb/tests/integration/test_real_width_contract.py and gelb/tests/integration/test_reproducibility.py, which also carry `requires_gpu_cuda_cpp_compiler`; the smoke test's CUDA case carries neither and skips through its own toolchain check.
- `smoke` is registered in gelb/tests/integration/conftest.py, which also adds `--run-smoke`, `--smoke-iters` (default 10) and `--smoke-backends` (default `serial`; also `cpu_highway` and `gpu_cuda_cpp`). Without `--run-smoke` every `smoke` test is skipped. gelb/tests/integration/test_examples_smoke.py compiles and runs every `.gelb` file under `examples/` on the selected backends.

6.4 The emission snapshot

scripts/emission_snapshot.py compiles every example under `examples/` with the three backends (`--num-timesteps 10 --write-every 10 --nc-output out.nc --seed 42`), stores the generated sources and writes a `MANIFEST.json` (the snapshot's own index) that records which examples fail to compile:

```
uv run python scripts/emission_snapshot.py snapshot -o /tmp/emis
# ... change the compiler ...
uv run python scripts/emission_snapshot.py compare  -o /tmp/emis
```

`compare` exits 1 on any byte difference, new failure, new pass or emission missing from the baseline; `diff-backends` needs no baseline and reports examples whose outcome differs between backends. CONTRIBUTING.md requires the snapshot to be identical for any backend work whose change to the generated code is not intentional and explained; a frontend or midware change that alters the AST handed to the backends changes the emission too.

6.5 The numeric suite

`uv run pytest -q gelb/tests/numeric` runs the battery described in gelb/tests/numeric/README.md and *Backends*: `test_generated_constants.py` needs only the oracle, `test_generated_runtime.py` builds the runtime cases on each available backend with `-O0` strict native flags and compares every cell bit for bit. gelb/tests/numeric/known_failures.py lists diagnosed expected failures; it is empty.

6.6 What CI runs

.github/workflows/release-health.yml, on every push and pull request, on `ubuntu-latest`: installs `g++`, `gfortran`, `libnetcdf-dev` and `libnetcdff-dev`; `uv sync --all-extras`; `uv build --wheel --sdist`; installs the wheel into a fresh venv and runs `gelbc examples/basic/1d/ex0_int_vec_setto_one.gelb --diagnostics-json` from it; then `uv run pytest` on gelb/tests/integration/test_example_diagnostics.py, gelb/tests/integration/test_geometry_lint.py, gelb/tests/unit/midware/test_periodic_continuity_validator.py and gelb/tests/numeric/; and finally `uv run sphinx-build -b html -W --keep-going docs /tmp/gelb-docs`, so a Sphinx warning fails the build. CI has no GPU, so the `gpu` cases skip there.

CHAPTER 7

Design decisions

Ten decisions a contributor needs to know in order not to undo them.

The generated parser is committed. `GeLBLexer.py`, `GeLBParser.py` and `GeLBVisitor.py` in gelb/antlr_part/generated_code/ are tracked so that the package installs and runs without a JDK (CONTRIBUTING.md). gelb/frontend/stage1_frontend.py falls back to `invoke_antlr4_tool` only when the import fails, and that function (gelb/utils/misc.py) writes into a scratch directory and replaces the live files only on success (gelb/tests/unit/test_antlr_regeneration.py). A grammar change and its regenerated files belong in one commit.

The midware simplifies only by identities that hold bit for bit. gelb/midware/ast_simplify.py lists what the two rewrite passes may do: `1*x`, `-1*x`, `x/1`, `-(-x)`, the zero identities only where the sign of a zero cannot change, sign moves that IEEE 754 defines, literal folding in the expression's width, and sharing of identical *ordered* subtrees. Nothing re-associates. The payoff is the *Default vs plain build* column of *What must agree*: a default and a plain build give the same bits (on CUDA, with nvcc's fusion off), which gelb/tests/numeric/test_generated_runtime.py checks on every backend by holding both builds to an independently compiled C++ oracle at zero ULP; its CUDA build passes `--fmad=false`. A rewrite that is "close enough" belongs behind a flag, not in the default passes.

The collinearity rewrite is off by default. gelb/midware/CollinearityRewriteVisitor.py turns `X - C*(X - Y)` into `(1-C)*X + C*Y`, a re-association without a general error bound. `PipelineOptions.enable_collinearity_rewrite` in gelb/compiler_pipeline.py defaults to `False` and gelb/gelbc.py always passes `False` and accepts `--no-collinearity-rewrite` without acting on it; the pass is never enabled from the command line. gelb/tests/unit/test_compiler_pipeline.py asserts that the pass runs only when enabled.

The storage layout is chosen per instruction set. `_auto_layout_choice` in gelb/backends/highway/layout.py resolves `--storage-layout auto`: SoA in 1D; in 2D and 3D the cell-block-tiled layout (CBT) when Highway, preprocessed by the C++ compiler with `GELB_CXXFLAGS` or else `-march=native`, selects one of its 512-bit targets (`_detect_avx512_available` in gelb/backends/highway/common.py, overridable with `GELB_HAS_AVX512`), SoA otherwise, because the tile shuffle assumes eight lanes and gives wrong results with fewer. The AVX-512 macros alone are not enough: Highway's default target selection also requires AES and PCLMUL, so GCC's bare `-march=x86-64-v4` selects two-lane SSSE3. The probe predicts gelb-run's build only when both use the same compiler and flags. Where they differ, a wrong yes stops at the assert below, and a wrong no costs only speed. The same probe makes `gelbc` reject `--storage-layout cbt`, and every CBT program carries a `static_assert` on the lane count, so a CBT program built for a narrower target fails to compile instead of running wrongly; do not remove either guard. The CUDA generator fixes `storage_layout='soa'`. Every layout gives the same bits, which gelb/tests/integration/test_reproducibility.py checks.

The three backends must agree. The table *What must agree* in docs/user/19_numerical_semantics.rst is the contract, and the chapter says that where the compiler disagrees with it, the compiler is wrong. The numeric battery, the tutorial regressions (gelb/tests/integration/test_tutorial_regressions.py) and the examples smoke test can run on all three backends, and `emission_snapshot.py diff-backends` fails when an example compiles on one backend and not on another. A feature is not done when one backend emits it.

The native compiler fuses nothing on the CPU; a strict build fuses nothing anywhere. The default `GELB_FFLAGS` and `GELB_CXXFLAGS` of gelb/scripts/gelb-run carry `-ffp-contract=off`; the one fusion GeLB decides itself is `gelb_mul_add`: the scalar helper in gelb/backends/cpp_runtime_common.hpp (`std::fma` where the including header defines `GELB_FMA_NATIVE`: `HWY_NATIVE_FMA` in the Highway header, `1` in the CUDA one) and its vector overload in gelb/backends/cpu_highway_runtime.hpp, both switched off once by `GELB_STRICT_FP`. A default CUDA build keeps nvcc's fusion; `--strict-fp` adds `-fmad=false`. gelb/tests/integration/test_real_width_contract.py checks the flags and the fusion probe.

The CUDA generator is a subclass of the Highway generator. `CudaCppCodeGenerator` in gelb/backends/stage3_backend_gpu_cuda_cpp.py inherits expression and statement emission from `HighwayCodeGenerator` and overrides the loops with kernels, so expression semantics are written once for both C++ backends; the Fortran generator is the independent second implementation.

``random()`` draws are sequenced before anything moves. gelb/midware/DrawSequencing.py runs before the inliner and the rewrites (`compile_source`), giving every draw its own temporary in source order, because C++ and Fortran leave the order of two calls in one expression unspecified and the language promises one draw per element, left to right.

A diagnostic is an exception that prints itself and exits. `GeLBException` in gelb/utils/exceptions.py prints the problem and the hint, with the source excerpt when the subclass supplies a location (`_source_location`), and calls `os._exit(1)`; both happen in `__str__`, which the constructor triggers with `print(self)`, except under pytest and in `--diagnostics-json` mode (`_suppress_print_and_exit`). A pass reports a problem by raising, never by printing, so the JSON contract of the VS Code extension holds.

Visitor dispatch is explicit. `ASTBaseVisitor.visit` (gelb/frontend/ASTBaseVisitor.py) raises `GeLBExMissingASTVisitationMethod` instead of skipping a node, so a new node class must be handled in every visitor that can reach it, and an unhandled one fails in the tests rather than in the output.